

Amendments to the CMTAT Distribution module

The contracts are **NOT audited**. Do not use them in production without an audit.

The CMTA framework functional specifications (June 2026) describe an optional **Distribution module** in section 3.2.4, with functionalities numbered 27 to 32 — see [cmtat-framework-functional-specifications-june-2026.pdf](#).

This document holds the **amendments**: changes to functionalities the specification already defines. Each one constrains or clarifies existing text, so a conforming implementation may already satisfy it — the specification simply does not say so, which is what lets two conforming implementations disagree.

The specification text these refer to

Quoted from section 3.2.4 of [cmtat-framework-functional-specifications-june-2026.pdf](#) so a proposal can be read without opening it. The PDF remains authoritative; square brackets are the specification's own.

3.2.4 Distribution module

Issuers may be required to make distributions to holders of securities (e.g. dividend payments for equity securities or interest payments for debt securities). Issuers may wish to carry out distributions off-chain (i.e. by transferring fiat currencies to the securities' holders' bank account). However, if the issuer intends to carry out such distributions on-chain, this may require the distribution of new tokens to existing token holders, on the basis of a snapshot carried out at the moment the legal entitlement to the distribution arises as not all token holders may be eligible for distributions.

Distribution events are typically performed according to a predefined schedule, and according to the token distribution at a given time, which can be determined by a snapshot performed with the Snapshot module.

Functionalities

27. **Distribution create parameters**: Define settlement token (i.e. the token that is to be distributed), identify a (past or future) block time/height for distribution snapshot, and amount to be distributed.
28. **Distribution set eligibility**: Flag a given users' tokens as being eligible or non-eligible to receive distributions (default: eligible).
29. **Distribution set deposit**: Send deposit amount for claiming settlement tokens to token holders flagged as eligible.
30. **Distribution claim deposit**: Allow token holders to claim their share of a deposit, identified by a deposit Identification, according to the token balance at the snapshot created at the defined time/height.

Additional use cases for tokens representing debt instruments:

31. **Distribution schedule**: Define a schedule for interest payments [and repayment of the par value at maturity], based on the token attributes.

32. **Distribution unschedule:** Cancel the previously set schedule for interest payments [and repayment of the principal amount at maturity].

C-6 and C-8 refer to the token attributes rather than to a functionality. Those are in section 3.1.1, under *Additional attributes applicable to tokens used for debt securities*:

- Currency of payments (if applicable)
- Par value (principal amount) (if applicable)
- Maturity date (if applicable)
- Interest rate (if applicable)
- Coupon payment frequency (if applicable)
- Interest schedule format (if applicable). The purpose of the interest schedule is to set, in the parameters of the smart contract, the dates on which the interest payments accrue.
 - Format A: start date/end date/period
 - Format B: start date/end date/day of period (e.g. quarter or year)
 - Format C: date 1/date 2/date 3/...
- Interest payment date (if different from the date on which the interest payment accrues)

The amendments

id	Amends	Proposal	Why — what implementing it exposed
C-1	27, 30	Require a distribution's record date to resolve against a balance source that has already fixed those balances, and require implementations to reject one that has not	A snapshot lookup for a time that was never scheduled does not fail — <code>SnapshotEngine</code> returns the holder's live balance (<code>_snapshotBalanceOf</code> ends <code>return snapshotted ? value : ownerBalance</code>). A distribution funded against a mistyped date therefore pays out pro-rata to balances <i>at claim time</i> , which anyone can change by acquiring tokens before claiming. The requirement is that the balances are fixed at the record date and cannot change afterwards , not that a snapshot exists — an off-chain pinning at a block height satisfies it equally. Phrasing it against a snapshot would forbid the alternatives in Eligibility without a snapshot ; phrasing it against a free timestamp permits the failure above.
C-2	30	Cap the claim period — a deadline after which a claim is refused	Functionality 30 says holders may claim; it never says until when. Without a deadline a distribution is an open liability forever, and the issuer can never close its books on a period. <code>IncomeVault</code> adds <code>timeLimitToWithdraw</code> .

id	Amends	Proposal	Why — what implementing it exposed
C-3	30	State the rounding direction for a holder's share	Pro-rata division always leaves dust. The specification is silent, so two conforming implementations can disagree on who gets it. State that shares round down , which makes the residue a known quantity rather than an accident.
C-4	28	Say whether eligibility is per distribution or per address, and when it is evaluated	A flag set in advance and a check performed at payout behave differently: an address frozen between the record date and the claim is eligible under one reading and not the other. <code>IncomeVault</code> evaluates at payout, through the same pause / freeze / RuleEngine path a transfer takes.
C-5	29	Forbid, or define, topping up a distribution whose claiming is already open	Nothing in 29 prevents a second deposit after holders have begun claiming. Those who already claimed took their share of the smaller amount; those who had not take a share of the larger. The specification should either forbid it or define the accounting.
C-6	27 + debt attributes	Reconcile the settlement token: per distribution, or per instrument?	Functionality 27 puts it per distribution; <code>DebtInstrument.currencyContract</code> puts it per instrument, and is already an <code>address</code> rather than a string. These are two different data models for the same thing, in one framework.
C-7	31, 32	Specify them as derived from the Snapshot and Debt modules , not as a store of their own	The dates already exist as <code>uint256[]</code> in the Snapshot module and the terms as strings in the Debt module. Saying so keeps one source of truth for record dates; implying a third store invites a schedule that disagrees with the snapshots the balances are actually read from.
C-8	Debt attributes	Give the schedule fields a machine-readable form , alongside the descriptive strings	<code>couponPaymentFrequency</code> , <code>interestScheduleFormat</code> and <code>interestPaymentDate</code> are prose. Any automation — a keeper funding coupons, a contract asserting the next payment date — must parse them off-chain and be trusted. An optional structured form would make 31 executable without removing the human-readable one.

id	Amends	Proposal	Why — what implementing it exposed
C-9	27, 29	Say whether the deposit must be held as the settlement token , and what <code>amount</code> means when that token is itself a vault share	Nothing constrains an implementation to hold the deposit idle, or the settlement token to be a plain ERC-20. Holding the float as ERC-4626 shares turns a fixed nominal obligation into one the contract may be unable to pay in full; distributing a vault share leaves "amount" ambiguous between shares and assets, with opposite rounding conventions on the two sides. See Holding a distribution deposit in an ERC-4626 vault .

C-1 and C-5 can cause value to move incorrectly. A record date resolved against balances that were never fixed pays out on balances anyone can still change (C-1); a distribution topped up after claiming has opened pays two holders different rates for the same entitlement (C-5). The remaining seven leave a question open rather than a door: they permit two conforming implementations to answer differently, which is a problem for an issuer comparing them and for a holder reading one.